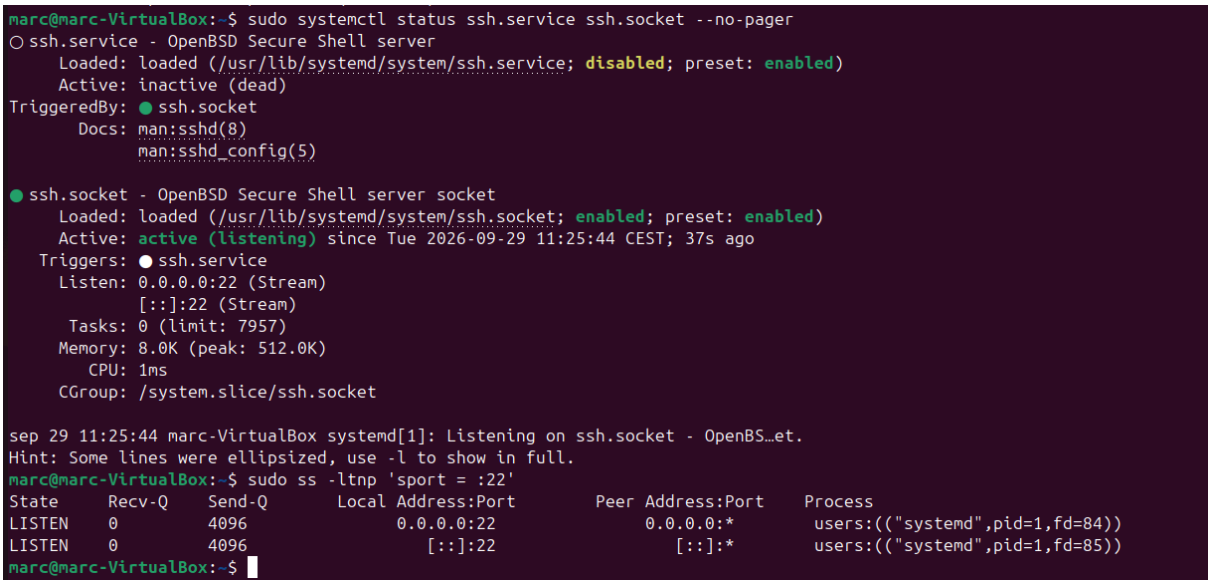
L1 · Necessitat, repositori i entorn

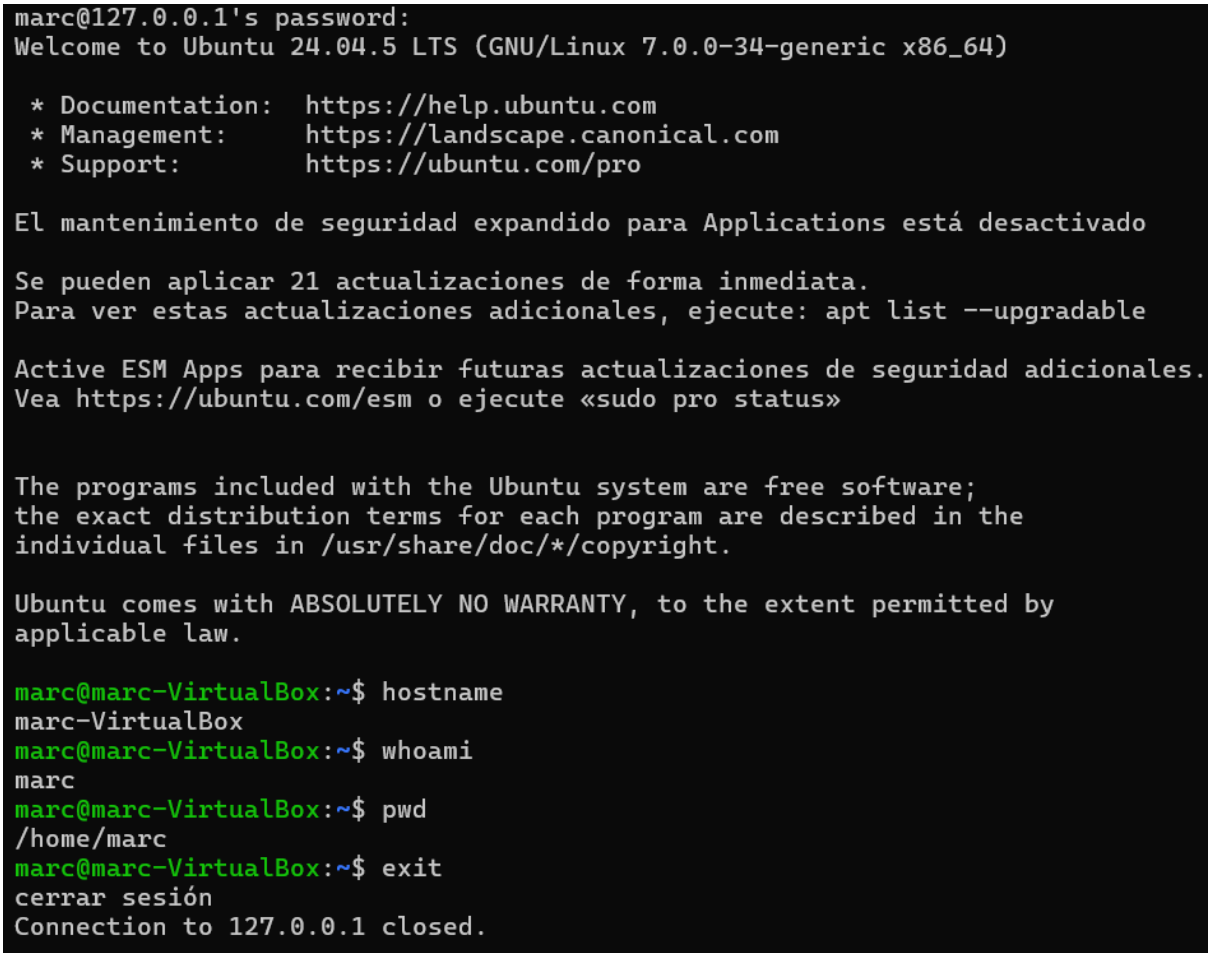
Desplega, configura i verifica Odoo

Estat i configuració de xarxa del servidor SSH

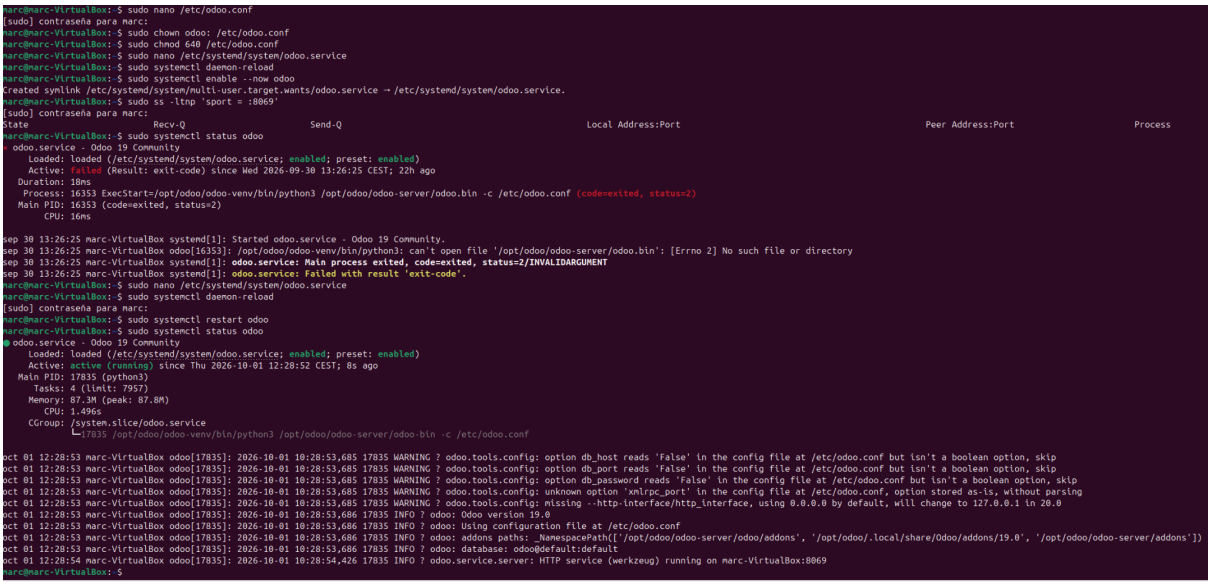


Pas 1. Instal·lació com a servei

La imatge mostra l'inici de sessió correcte a la màquina virtual Ubuntu Server a través del protocol SSH des de l'equip amfitrió.



Aquesta mostra la interacció amb el gestor de serveis de Linux (`systemd`). Es pot observar un primer intent fallit on el servei no arranca per problemes amb la ruta de l'executable o del fitxer de configuració. Després d'aplicar correccions en la configuració i recarregar el dimoni (`sudo systemctl daemon-reload`), s'executa una segona comprovació (`sudo systemctl status odoo`) on s'evidencia que el servei `odoo.service` finalment es troba en estat actiu i funcionant (active (running)). El registre també mostra els paràmetres de connexió a la base de dades i que el servidor web s'ha iniciat correctament pel port 8069.



Per validar que el servei Odoo instal·lat de forma nativa està escoltant peticions correctament, s'executa la comanda:



Pas 2. Segon tipus de desplegament

Fase 1: Preparació de l'entorn

La imatge documenta la configuració dels permisos d'usuari per a Docker a la màquina virtual. Mitjançant la comanda `sudo usermod -aG docker $USER`, s'afig l'usuari actual (`marc`) al grup de sistema `docker`, el que permetrà executar contenidors en el futur sense necessitat d'utilitzar el prefix `sudo` de forma constant. Seguidament, s'aplica el canvi de grup en la sessió activa amb `newgrp docker`. Finalment, es comproven les versions de les ferramentes instal·lades

```
marc@marc-VirtualBox:~$ sudo usermod -aG docker $USER
[sudo] contraseña para marc:
marc@marc-VirtualBox:~$ newgrp docker
marc@marc-VirtualBox:~$ docker --version
Docker version 29.9.0, build f415da8
marc@marc-VirtualBox:~$ docker compose version
Docker Compose version v5.6.0
marc@marc-VirtualBox:~$
```

Aquesta evidència documenta la creació de l'estructura de directoris necessària per al projecte. Mitjançant la comanda `mkdir ~/odoo-docker` es crea la carpeta principal de treball. Posteriorment, i ja dins del directori, l'execució de `mkdir -p {config,addons,logs,backups}` genera de forma simultània les subcarpetes requerides per a mapejar els volums persistents dels contenidors cap al disc de la màquina virtual. L'eixida de la comanda `ls -la` confirma que totes les carpetes s'han creat satisfactòriament amb els permisos adequats.

```
marc@marc-VirtualBox:~$ mkdir ~/odoo-docker
marc@marc-VirtualBox:~$ cd ~/odoo-docker
marc@marc-VirtualBox:~/odoo-docker$ mkdir -p {config,addons,logs,backups}
marc@marc-VirtualBox:~/odoo-docker$ ls -la
total 24
drwxrwxr-x  6 marc docker 4096 oct 14 19:48 .
drwxr-x--- 16 marc marc   4096 oct 14 19:47 ..
drwxrwxr-x  2 marc docker 4096 oct 14 19:48 addons
drwxrwxr-x  2 marc docker 4096 oct 14 19:48 backups
drwxrwxr-x  2 marc docker 4096 oct 14 19:48 config
drwxrwxr-x  2 marc docker 4096 oct 14 19:48 logs
marc@marc-VirtualBox:~/odoo-docker$
```

Fase 2: Configuració de Docker Compose

La imatge documenta el contingut de l'arxiu docker-compose.yml. Aquest fitxer defineix l'arquitectura de contenidors necessària per al projecte:

- Un servei **db** basat en la imatge de PostgreSQL, amb la definició de variables d'entorn per a l'usuari i la contrasenya (odoo1234), a més d'una comprovació de salut (healthcheck) i el mapeig del volum per a garantir la persistència de la base de dades.
- Un servei **web** basat en la imatge oficial d'Odoo 16.0, configurat per a dependre (depends_on) de l'estat saludable de la base de dades abans d'arrancar. També s'hi mostren les variables d'entorn de connexió, el mapeig de ports (8069:8069) per a l'accés des del navegador, i la declaració de volums interns per a guardar la configuració, complements (addons) i registres (logs).

```yaml
services:
  db:
    image: postgres:latest
    container_name: odoo_postgres
    enviroment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=odoo
      - POSTGRES_PASSWORD=odoo1234
      - PGDATA=/var/lib/postgresql/data/pgdata
    volumes:
      - odoo-db-data:/var/lib/postgresql/data
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U odoo"]
      interval: 30s
      timeout: 5s
      retries: 3

  web:
    image: odoo:16.0
    container_name: odoo_app
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "8069:8069"
    enviroment:
      - HOST=db
      - USER=odoo
      - PASSWORD=odoo123
    volumes:
      - odoo-web-data:/var/lib/odoo
      - ./config:/etc/odoo
      - ./addons:/mnt/extra-addons
      - ./logs:var/log/odoo
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8069/web/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 60s

volumes:
  odoo-web-data:
  odoo-db-data:

networks:
  default:
    name: odoo_network
```

La imatge mostra el contingut del fitxer odoo.conf, el qual estableix els paràmetres principals d'execució del servidor Odoo. Sobserva:

- **Connexió a la base de dades:** Les credencials (odoo123), el nom d'usuari (odoo) i el *host* (db), que coincideixen amb els valors definits al servei de PostgreSQL en el docker-compose.yml.
- **Configuració de xarxa i proxy:** El port de connexió XML-RPC fixat al 8069 i el paràmetre proxy_mode = True.
- **Rutes de directoris:** L'especificació de la ruta on es troben els mòduls addicionals (addons_path), el directori de dades (data_dir) i la ubicació i nivell del fitxer de registre d'esdeveniments (logfile i log_level).
- **Límits de recursos i contrasenya d'administrador:** S'estableixen els límits de memòria i temps de CPU per a evitar que el contenidor gaste tots els recursos de la màquina host, així com la contrasenya de l'administrador per a la gestió de bases de dades.

```
[options]
db_host = db
db_port = 5432
db_user = odoo
db_password = odoo123
db_maxconn = 64

xmlrpc_port = 8069
xmlrpc_interface = 0.0.0.0
proxy_mode = True

addons_path = /usr/lib/python3/dist-packages/odoo/addons,/mnt/extra-addons
data_dir = /var/lib/odoo

logfile = /var/log/odoo/odoo.log
log_level = info
logrotate = True

admin_passwd = admin123
list_db = True
workers = 0
limit_memory_hard = 2684354560
limit_memory_soft = 2147483648
limit_time_cpu = 600
limit_time_real = 1200

without_demo = False
```

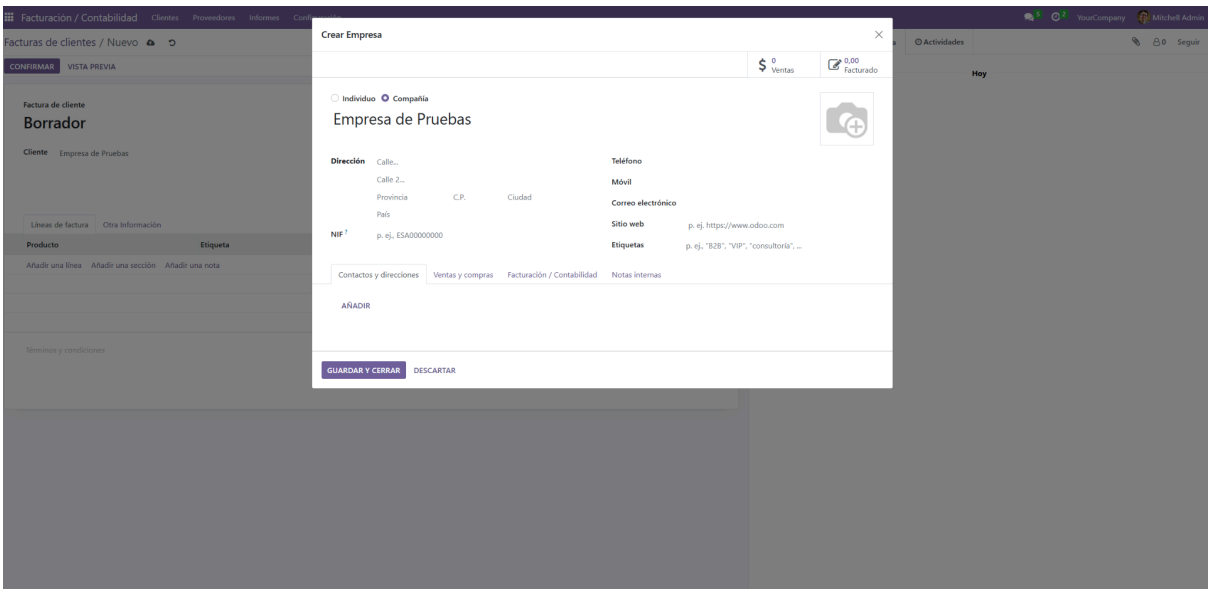
Fase 3: Desplegament i verificació

Aquesta evidència demostra l'èxit en el desplegament de la infraestructura Docker. Primer, s'executa `docker compose pull` per a descarregar les imatges oficials d'Odoo 16 i PostgreSQL. Seguidament, amb la comanda `docker compose up -d`, es creen els volums persistents i s'arranquen els contenidors en segon pla. Finalment, la comprovació de l'estat mitjançant `docker compose ps` verifica que tant el contenidor `odoo_app` com el `odoo_postgres` estan actius ("Up") i en estat saludable ("healthy"), amb els ports de connexió correctament assignats.

```
marc@marc-VirtualBox:~/odoo-docker$ docker compose pull
[+] pull 2/2
 ✔ Image odoo:16.0       Pulled
 ✔ Image postgres:latest Pulled
marc@marc-VirtualBox:~/odoo-docker$ docker compose up -d
[+] up 5/5
 ✔ Network odoo_network              Created
 ✔ Volume odoo-docker_odoo-db-data   Created
 ✔ Volume odoo-docker_odoo-web-data  Created
 ✔ Container odoo_postgres           Healthy
 ✔ Container odoo_app                Started
marc@marc-VirtualBox:~/odoo-docker$ docker compose ps
NAME            IMAGE             COMMAND                  SERVICE   CREATED         STATUS                   PORTS
odoo_app        odoo:16.0         "/entrypoint.sh odoo"    web       5 minutes ago   Up 5 minutes (healthy)   0.0.0.0:8069->8069/tcp, [::]:8069->8069/tcp, 8071-8072/tcp
odoo_postgres   postgres:latest   "docker-entrypoint.s…"   db        5 minutes ago   Up 5 minutes (healthy)   5432/tcp
marc@marc-VirtualBox:~/odoo-docker$
```

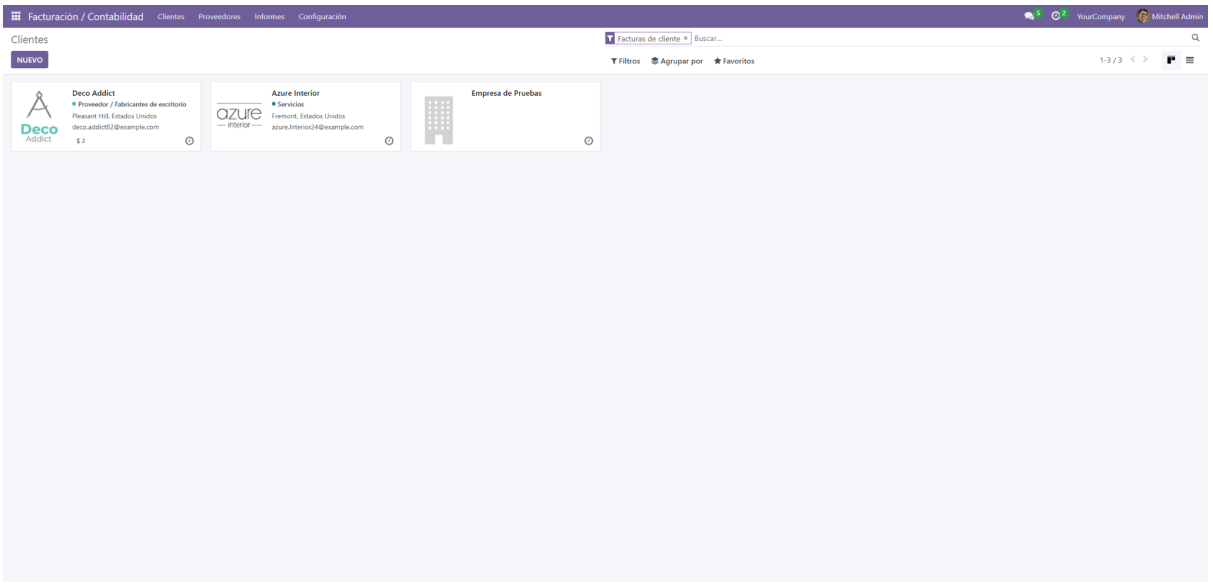
Demostració de la persistència de dades

La imatge mostra la creació d'una nova companyia fictícia anomenada "Empresa de Pruebas" des del mòdul de Facturació, per a comprovar que els volums persistents de Docker s'han configurat correctament. Una vegada guardat aquest registre a la base de dades, es procedirà a parar i destruir els contenidors (`docker compose down`) per a validar si la informació sobreviu en reiniciar el servei.



```
marc@marc-VirtualBox:~/odoo-docker$ docker compose down
[+] down 3/3
 ✔ Container odoo_app       Removed
 ✔ Container odoo_postgres  Removed
 ✔ Network odoo_network     Removed
marc@marc-VirtualBox:~/odoo-docker$ docker compose up -d
[+] up 3/3
 ✔ Network odoo_network     Created
 ✔ Container odoo_postgres  Healthy
 ✔ Container odoo_app       Started
marc@marc-VirtualBox:~/odoo-docker$ S
```

Aquesta evidència demostra l'èxit en la configuració dels volums persistents. Després d'haver parati destruit els contenidors de l'entorn amb `docker compose down` i haver-los tornat a alçar, s'accedeix novament a la llista de clients dins del mòdul de Facturació. La imatge mostra que el registre fictici "Empresa de Pruebas", continua existint a la base de dades junt amb la resta d'informació. Açò confirma que la informació de PostgreSQL s'ha guardat de forma permanent a l'emmagatzematge de la màquina virtual.



Fase 4: Monitorització de l'entorn

Aquesta evidència documenta l'execució de l'script personalitzat de control (`./scripts/monitor.sh`). Agrupa l'estat de salut actual dels contenidors (`ESTAT DELS CONTENIDORS`), el consum detallat de CPU, memòria RAM i trànsit de xarxa en temps real (`ÚS DE RECURSOS`), l'ocupació física a l'emmagatzematge de la màquina (`ESPAI EN DISC`) i una validació HTTP contra el healthcheck intern d'Odoo que retorna el missatge `{"status": "pass"} Odoo OK` (`CONNEXIONS ACTIVES`). Amb açò es certifica que l'entorn de desenvolupament és estable i opera dins d'uns paràmetres òptims.

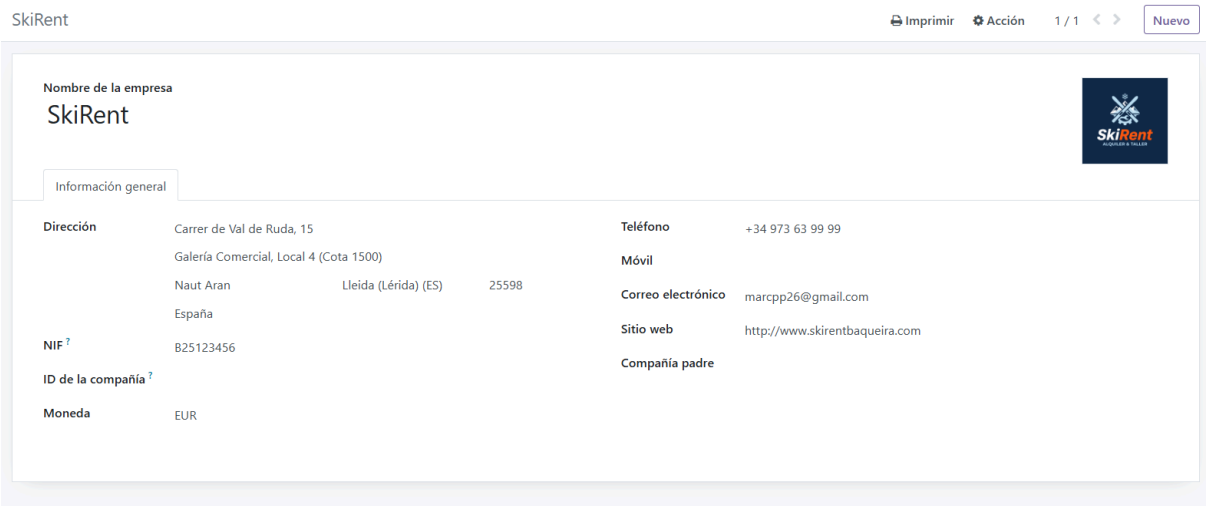
```
marc@marc-VirtualBox:~/odoo-docker$ ./scripts/monitor.sh
=== ESTAT DELS CONTENIDORS ===
NAME            IMAGE             COMMAND                  SERVICE   CREATED        STATUS                    PORTS
odoo_app        odoo:16.0         "/entrypoint.sh odoo"    web       42 hours ago   Up 10 minutes (healthy)   0.0.0.0:8069->8069/tcp, [::]:8069->8069/tcp, 8071-8072/tcp
odoo_postgres   postgres:latest   "docker-entrypoint.s…"   db        42 hours ago   Up 10 minutes (healthy)   5432/tcp

=== ÚS DE RECURSOS ===
CONTAINER                                                          CPU %     MEM USAGE / LIMIT     NET I/O
2cd99fe0bf830901a6bb600d2b2f452c287af74d9f75437998e393f951d24236   0.03%     219.6MiB / 7.751GiB   367kB / 365kB
1b3f3d955cb9ed8339d0bebac9384ca08c9638baca30b1144dee6800c1d81c32   0.04%     118.9MiB / 7.751GiB   378kB / 354kB

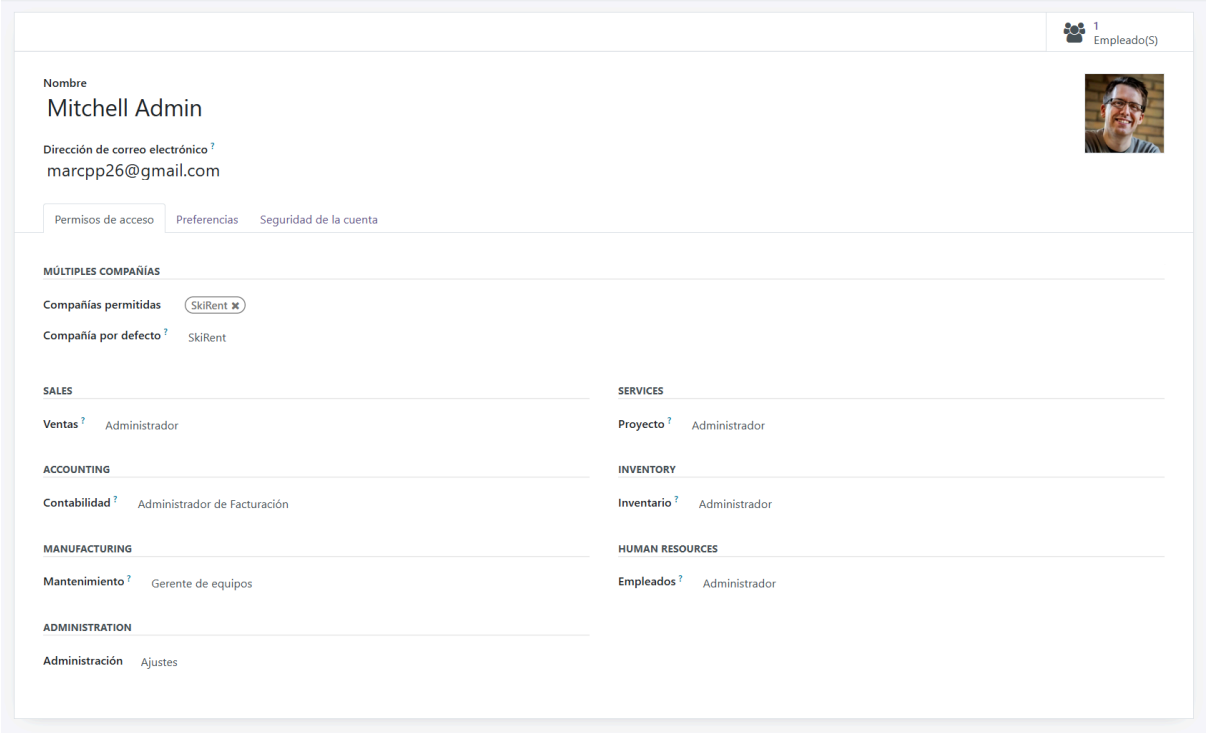
=== ESPAI EN DISC ===
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          2         2         3.233GB   0B (0%)
Containers      2         2         28.67kB   0B (0%)
Local Volumes   4         3         202.2MB   0B (0%)
Build Cache     0         0         0B        0B

=== CONNEXIONS ACTIVES ===
{"status": "pass"} Odoo OK
marc@marc-VirtualBox:~/odoo-docker$
```

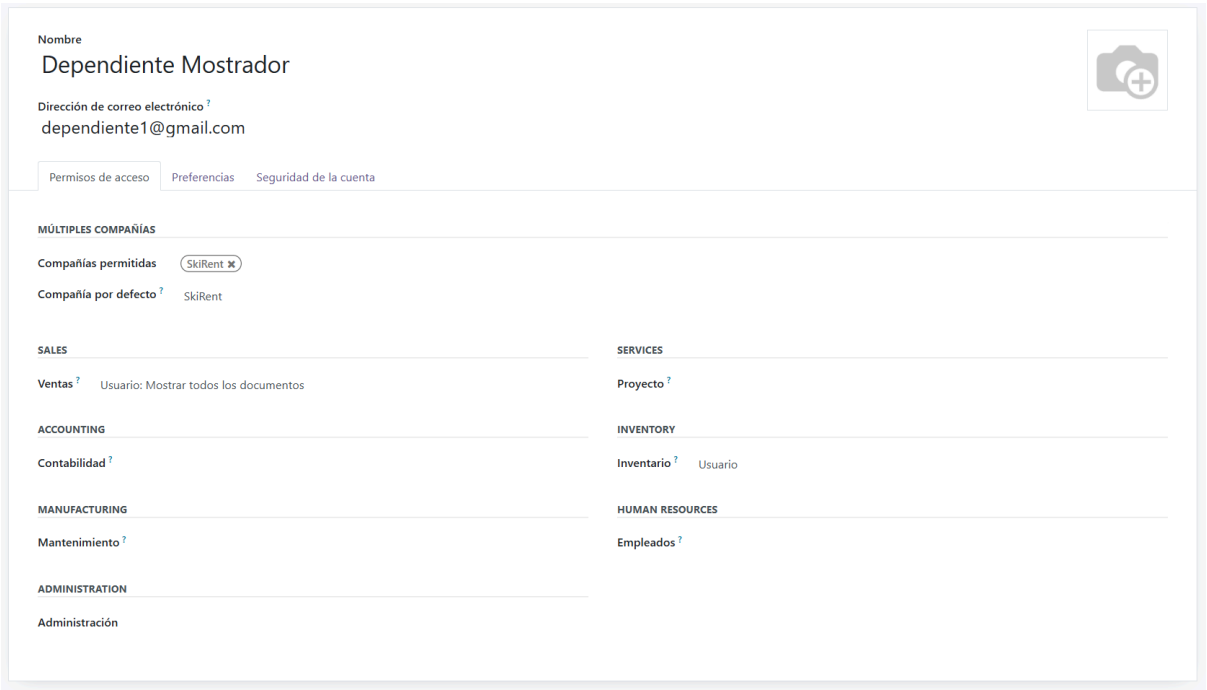
Pas 3. Adaptació empresarial

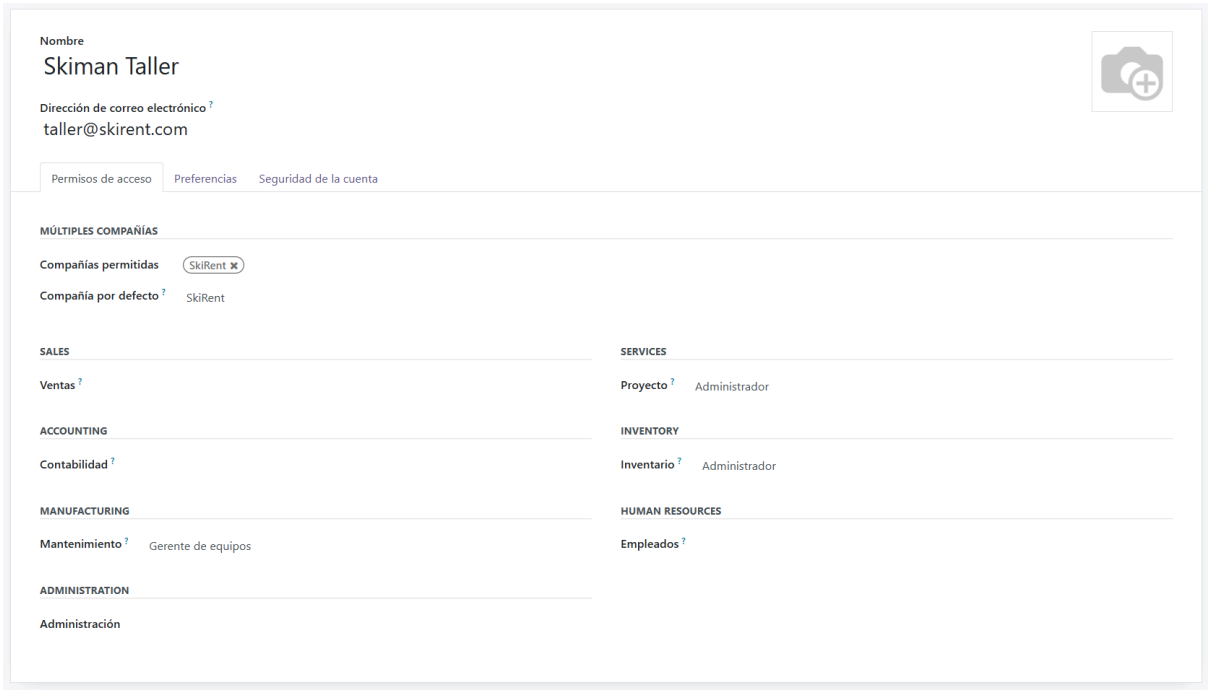


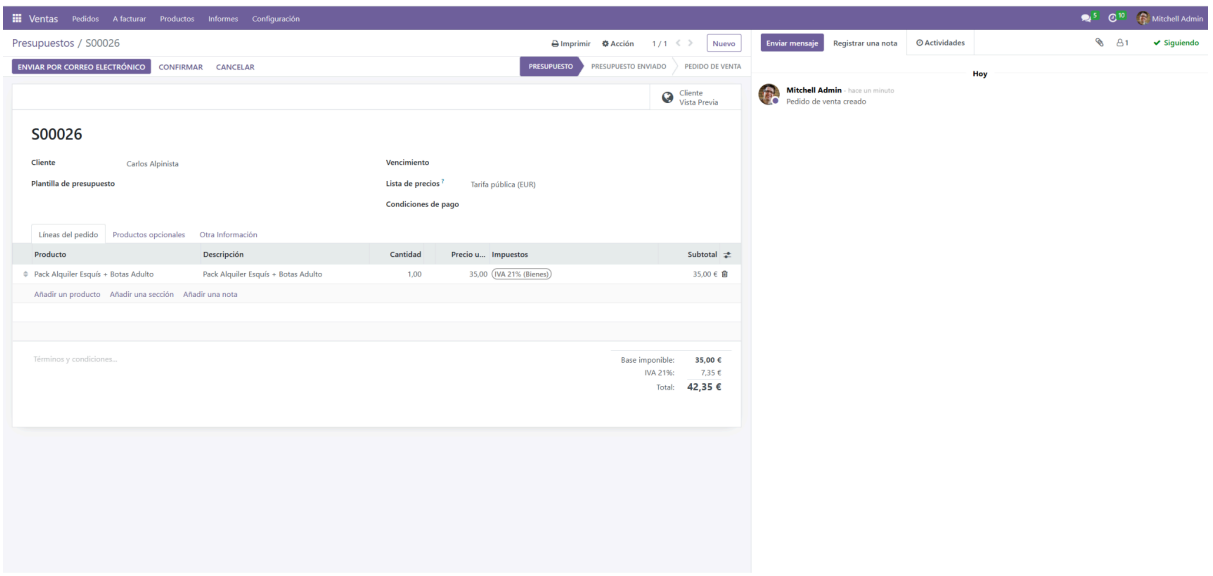
Representa la Variant 1 de Gestió Bàsica, on s'atorguen privilegis d'Administrador global sobre tots els mòduls instal·lats. Aquest model es justifica operativament per als períodes de temporada baixa, on un únic treballador assumeix tant l'atenció al client com les tasques tècniques al taller i la facturació.



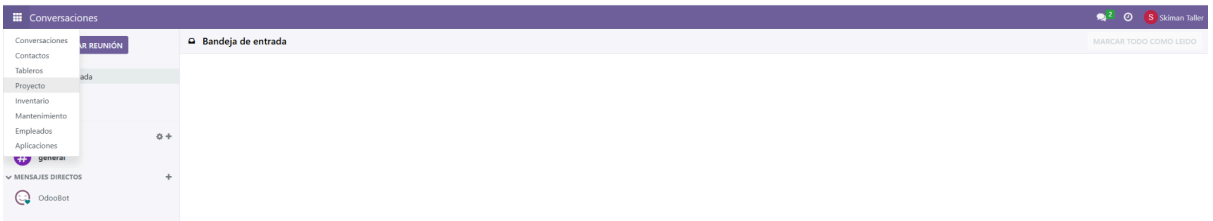
Per a la segona variant de gestió separada per rols, s'ha aplicat una segregació de funcions (SoD) dividint la platilla en dos rols diferenciats, el qual es esencial durant la temporada alta d'esquí. El rol Depenent Mostrador té permisos de bentes per a gestionar contrats i reserves, però no pot alterar la configuració del almacén. El rol Skiman Taller té control total sobre l'inventari i el manteniment per a gestionar les reparacions, però no té l'accés a ventes.



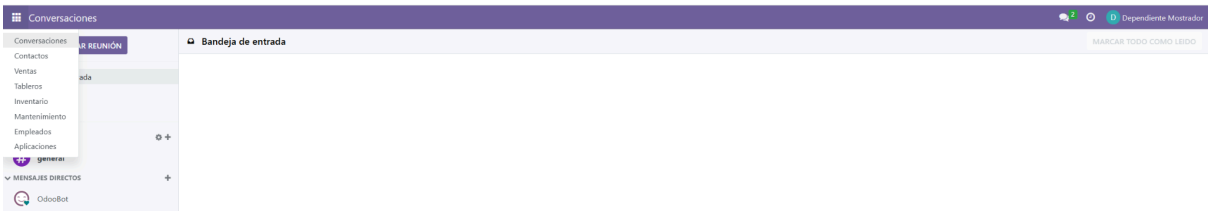




S'observa que aquest perfil té accés als mòduls de Projecte, Inventari i Manteniment, necessaris per a reparar el material, però se li ha revocat per complet l'accés al mòdul de Vendes per a protegir les dades financeres dels clients.

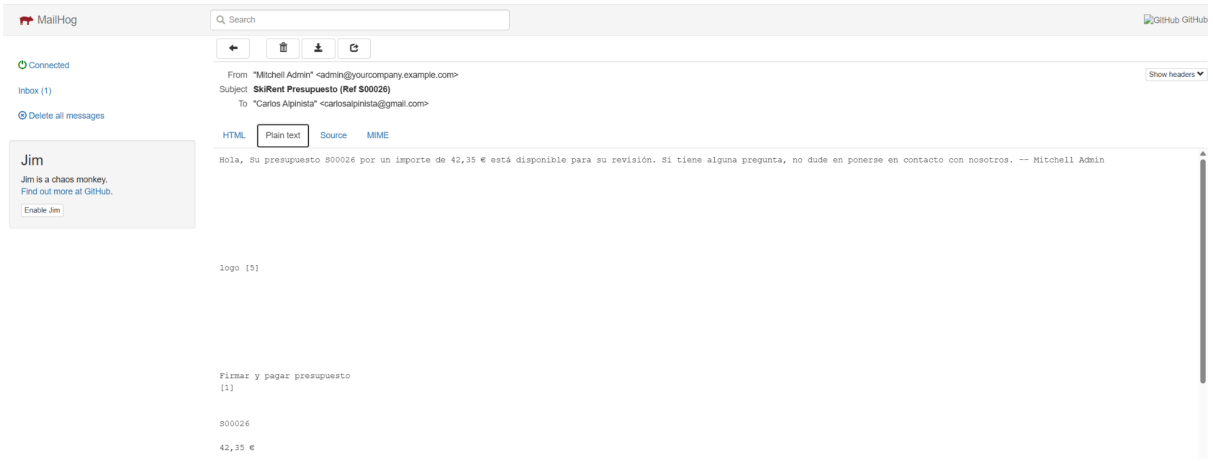


Aquest empleat sí que té accés a ventes per a generar contractes de lloguer, però té els permisos limitats en l'àrea d'inventari i configuració global.



Pas 4. Correu i incidència

Aquesta evidència demostra la correcta integració del servei MailHog dins de l'entorn de desenvolupament. Després de configurar el servidor SMTP a Odoo per a utilitzar el contenidor de MailHog (pel port 1025), s'ha enviat un pressupost de prova des del mòdul de Vendes ("SkiRent Presupuesto Ref S00026") destinat al client fictici "Carlos Alpinista".



Aquesta evidència documenta la simulació d'una fallada de connexió entre Odoo i el servidor de correu local. S'ha modificat intencionadament el paràmetre "Port SMTP" en la configuració del servidor d'eixida, assignant el port erroni `1026` en lloc del correcte (`1025`). En executar la prova de connexió, el sistema llança una finestra emergent d'Error d'usuari indicant que la prova ha fallat amb el missatge `111 Connection refused`. Aquest error registra una pèrdua temporal del servei de correu, ja que l'assignació d'un port erroni impedeix la correcta resolució de les peticions cap al contenidor de MailHog

